

Математические основы алгоритмов, осень 2020 г.

Лекция 8: Сжатие данных: метод Лемпеля–Зива LZ77 и LZ78, преобразование Берроуза–Вилера*

Александр Охотин

8 июня 2021 г.

Содержание

1 Сжатие данных, основанное на повторяющихся подстроках	1
1.1 Метод Лемпеля–Зива LZ77	1
1.2 Метод Лемпеля–Зива LZ78	2
1.3 Преобразование Берроуза–Вилера	4

1 Сжатие данных, основанное на повторяющихся подстроках

Метод Хаффмана и арифметическое кодирование основаны на том, что разные символы могут встречаться с существенно разной частотой. Но во многих текстах есть более интересная закономерность: какие-то подстроки встречаются чаще других. Это нужно использовать!

Наивный метод — посчитать число повторений всякой подстроки через суффиксное дерево, потом каким-то образом извлечь из этого набора данных не очень большую таблицу часто встречающихся подстрок, а потом эту таблицу ещё придётся где-то хранить. Получится нагромождение структур данных и алгоритмов, эффективность которого основана на непонятно каких эвристиках.

1.1 Метод Лемпеля–Зива LZ77

Метод Лемпеля–Зива — это простой, чёткий и легко реализуемый алгоритм сжатия данных, использующий повторяемость подстрок. Его главная мысль чрезвычайно проста: всякий раз, когда ранее прочитанная подстрока встречается повторно, алгоритм пишет вместо неё ссылку на её предыдущее вхождение.

Алгоритм LZ77 (Зив и Лемпель [1977]): на каждом шаге выводится тройка (d, ℓ, a) , что означает: сперва повторяется подстрока длины ℓ , ранее встречавшаяся d символов назад, а потом идёт символ a . Допускается $\ell > d$, в этом случае кодируется подстрока с периодическим окончанием. В конкретных реализациях всё это представляется по-разному, иногда разделяются (d, ℓ) и a .

Пример 1. Строка $w = abaababa$, её код — $(0, 0, a)(0, 0, b)(2, 1, a)(3, 2, b)(0, 0, a)$. Или так: $ab(2, 1)(3, 3)(2, 2)$.

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.

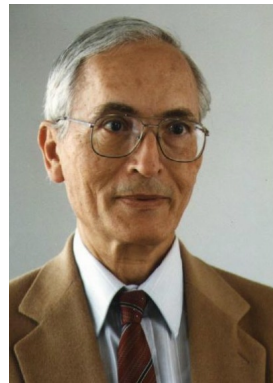


Рис. 1: Авраам Лемпель (род. 1936), Яков Зив (род. 1931).

У алгоритма LZ77 есть определённая свобода выбора ранее виденной подстроки на каждом шаге сжатия. Оказывается, что жадная стратегия, выбирающая на каждом шаге самую длинную такую подстроку, даёт наилучшее сжатие. Следующая теорема приводится без доказательства.

Теорема 1. *Жадный LZ77 оптимален.*

Самая простая реализация LZ77 — через суффиксное дерево, в котором на каждом шаге читаются следующие символы входной строки, пока не находится самая длинная раньше встречавшаяся подстрока. Недостатки: во-первых, если строка длинная, то сколь бы эффективные структуры данных мы ни использовали, алгоритм неизбежно начнёт замедляться. Во-вторых, будут находиться очень старые подстроки, и потому количество битов, необходимое для записи d , будет расти; придётся придумывать, как сжимать d , и алгоритм усложнится.

«Скольльзящее окно». Обычно используется вариант этого алгоритма, потенциально сжимающий немного хуже, однако более удобный в реализации. Подстроки ищутся среди последних m прочитанных символов — в «скользящем окне» (sliding window). Более старые символы будут забываться. Чтобы искать их эффективно, тоже можно воспользоваться суффиксным деревом — но для этого нужно научиться удалять из него все суффиксы длиннее m . Для этого в недостроенном суффиксном дереве, создаваемом алгоритмом Укконена, надо на каждом шаге стирать самый длинный суффикс за совокупное линейное время. Алгоритм будет помнить лист, соответствующий самому длинному суффиксу, и на каждом шаге будет удаляться дуга, ведущая в этот лист; после этого алгоритм, следуя по суффиксной ссылке, будет запоминать следующий по порядку суффикс. Если же самый длинный недостроенный суффикс находится как раз на этой дуге, то дуга будет делиться.

1.2 Метод Лемпеля–Зива LZ78

Ранее рассмотренный метод LZ77 основан на том, что повторяющиеся подстроки представляются в виде ссылок на свои предыдущие вхождения. Таким образом, удаётся обойтись без составления таблицы часто встречающихся подстрок.

Другой похожий алгоритм, LZ78 (Зив и Лемпель [1978]), строит-таки таблицу часто встречающихся подстрок — «словарь» (dictionary) — но делает он это по мере чтения строки. Словарь хранится в префиксном дереве, что позволяет легко находить самое длинное

продолжение входной строки, уже присутствующее в словаре. При декодировании строится в точности этот же словарь. В сжатом представлении строки словарь не хранится.

В начале работы в словаре содержится единственный элемент под номером ноль: $T_0 = \varepsilon$; иными словами, префиксное дерево состоит из корня, помеченного номером 0. На каждом шаге читается самая длинная строка $T_j = v$, уже имеющаяся в словаре, и выводится её код j ; также читается и выводится следующий символ a . При этом в словарь добавляется новая строка va — конкатенация только что прочитанной со следующим входным символом.

На префиксном дереве это выглядит так: после вывода очередной пары (j, a) алгоритм переходит в корень префиксного дерева, и дальше читает столько входных символов, сколько возможно. Когда очередной символ прочитать нельзя, создаётся новый лист, при этом выводится номер предыдущей вершины и прочитанный символ.

Пример 2. Строка $w = ababaabb$, в таблице $T_0 = \varepsilon$.

Кодирование:

Читается a , выводится $0a$, добавляется $T_1 = a$.

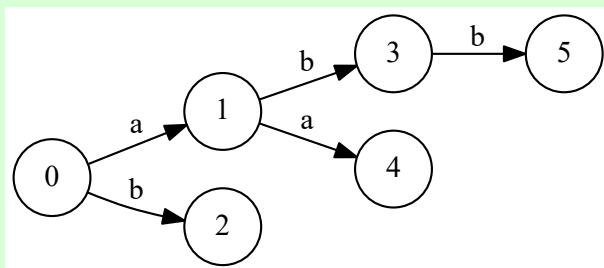
Читается b , выводится $0b$, добавляется $T_2 = b$.

Читается ab , выводится $1b$, добавляется $T_3 = ab$.

Читается aa , выводится $1a$, добавляется $T_4 = aa$.

Читается abb , выводится $3b$, добавляется $T_5 = abb$.

Код — $0a0b1b1a3b$.



Декодирование:

Читается $0a$, выводится a , добавляется $T_1 = a$.

Читается $0b$, выводится b , добавляется $T_2 = b$.

Читается $1b$, выводится ab , добавляется $T_3 = ab$.

Читается $1a$, выводится aa , добавляется $T_4 = aa$.

Читается $3b$, выводится abb , добавляется $T_5 = abb$.

При декодировании строится то же самое префиксное дерево. Алгоритму при этом потребуется находить строку по номеру в таблице: можно, например, завести для этого отдельный массив строк, или же читать в префиксном дереве путь из вершины в корень. Прочитав очередную пару (j, a) , алгоритм выводит строку T_j и символ a , после чего сразу перескакивает в вершину j и присоединяет к ней новый лист, с переходом по символу a .

В практических реализациях словарь имеет ограниченный размер (например, 2^{12} элементов), и когда он заполняется, есть несколько возможностей. Во-первых, можно продолжать кодирование с имеющимся словарём. Во-вторых, можно очистить словарь. Наконец, есть варианты алгоритма, которые при переполнении словаря начинают удалять из него наименее нужные строки (тем или иным образом выбираемые).



Рис. 2: Майкл Берроуз (род. 1963), Дэвид Вилер (1927–2004).

Также при реализации никогда не вредно добавить в конце арифметическое кодирование: можно на каждом шаге оценивать вероятности разных подстрок и выполнять один шаг арифметического кодирования с этими вероятностями.

Существует множество разнообразных вариантов метода Лемпеля–Зива — таких, например, как полученный Велчем [1984] алгоритм Лемпеля–Зива–Велча (LZW) — вариант алгоритма LZ78.

1.3 Преобразование Берроуза–Вилера

Burrows–Wheeler transform (BWT), см. Берроуз и Вилер [1994].

Берутся все циклические сдвиги строки $w = a_1 \dots a_n$. Они сортируются лексикографически, получается таблица T размера $n \times n$, где в каждой строчке — один из циклических сдвигов. Наконец, берётся последовательность последних символов $b_1 \dots b_n$ в таблице T и номер t строки w в этой таблице.

Ради чего всё это, если n битов переводятся в $n + \log_2 n$ битов? Если в исходной строке какая-то подстрока ai часто повторялась, то есть много циклических сдвигов, начинающихся с i и заканчивающихся на a , и после сортировки они окажутся рядом. Поэтому в преобразованной строке будут длинные последовательности повторяющихся символов. Использование в качестве метода сжатия данных: преобразовать, а потом применять другие методы к преобразованной строке — например, RLE.

Пример 3 (BWT). Пусть дана строка $w = abaababa$.

Все циклические сдвиги:

abaababa
baababaa
aababaab
ababaaba
babaabaa
abaabaab
baabaaba
aabaabab

Отсортированные:

aabaabab
aababaa
abaabaab
abaababa ✓
ababaaba
baabaaba
baababaa
babaabaa

Преобразование: $w' = bbbaaaaa$, строчка номер 4. Тогда RLE даст на выходе *bb1aa3*.

Утверждается, что по паре (w', t) восстанавливается исходная строка w . Проще всего показать, что можно восстановить всю таблицу T по её последнему столбцу $w' = b_1 \dots b_n$.

	b_1	
a_{n-1}	a_n	t
a_n	a_{n-1}	t'
	b_n	

Рис. 3: Перестановка p , дающая по номеру строки t номер строки $t' = p(t)$, содержащей следующий по порядку сдвиг.

Сперва последовательность $b_1 \dots b_n$ сортируется; и так как это перестановка символов исходной строки, получается в точности первый столбец $c_1 \dots c_n$ таблицы T .

$$\begin{pmatrix} c_1 & ? & \dots & ? & b_1 \\ \vdots & & \ddots & & \vdots \\ c_n & ? & \dots & ? & b_n \end{pmatrix}$$

В частности, вслед за каждым символом b_i в исходной строке идёт символ c_i , и потому $b_i c_i$, для всех i — это все подстроки длины 2 в исходной строке. Если их отсортировать, получатся первые два столбца таблицы T . К ним опять можно приписать столбец $b_1 \dots b_n$, отсортировать, и получить первые три столбца таблицы T . И так далее, можно восстановить всю отсортированную таблицу. Это доказывает обратимость преобразования, но ещё не даёт эффективного алгоритма для вычисления обратного преобразования.

Лемма 1. *BWT можно вычислить за линейное время.*

Доказательство. Строится суффиксное дерево для $w\$$. Затем дерево обходится, и лексикографическом порядке рассматриваются $n = |w|$ суффиксов строго большей длины, чем $w\$$. Первые n символов каждого суффикса — это один из циклических сдвигов строки w . При обходе для каждого суффикса выводится его n -й символ — это и получается последний столбец отсортированного массива циклических сдвигов. Время: $O(n)$. $\square$

Для обратного преобразования готовой структурой данных не обойтись, однако существует хитроумный алгоритм, позволяющий выполнить его за линейное время, причём с очень небольшой константой.

Лемма 2. *Обратное к BWT преобразование можно вычислить за линейное время.*

Доказательство. Главная мысль состоит в том, чтобы вычислить перестановку $p: \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, которая по номеру строчки i в таблице T даст номер следующего по порядку циклического сдвига. Зная последний столбец $b_1 \dots b_n$ таблицы T и номер t искомой строчки таблицы, можно сразу записать последний символ исходной строки: $a_n = b_t$. Далее, применяя к номеру строчки t функцию p , достаточно записать последние символы всех строчек, и это будут символы исходной строки, от конца к началу: $a_{n-i} = b_{p^i(t)}$.

Значения искомой перестановки p можно вычислить по следующей формуле.

Утверждение. $p(i) = 1 + |b_1 \dots b_{i-1}|_{b_i} + \sum_{a < b_i} |b_1 \dots b_n|_a$

Действительно, пусть i -я строчка таблицы T имеет вид ub . Нужно найти, какое место занимает в таблице T следующий по порядку циклический сдвиг bu . Какие циклические сдвиги будут раньше? Во-первых, все те, которые начинаются с символов, меньших, чем b ; их всего $\sum_{a < b_i} |b_1 \dots b_n|_a$. Что касается строк вида bv , начинающихся с того же символа b , строка bu занимает среди них в точности то же место, что строка ub занимает среди строк вида vb (поскольку $bv < bu$ тогда и только тогда, когда $vb < ub$). Поэтому достаточно посчитать порядковый номер строки ub среди всех строк, заканчивающихся на b — и этот номер задаётся выражением $|b_1 \dots b_{i-1}|_{b_i}$. Наконец, если строки нумеруются, начиная с единицы, то эта единица прибавляется.

Как вычислить все эти значения за линейное время? Создаётся массив чисел C_a , для всех символов $a \in \Sigma$, и делается первый проход по строке $b_1 \dots b_n$, во время которого, после чтения очередного символа b_i каждая переменная C_a принимает значение $|b_1 \dots b_i|_a$. Тогда в начале каждой i -й итерации значение $|b_1 \dots b_{i-1}|_{b_i}$ находится в переменной C_{b_i} , и можно покамест записать его в переменную $p(i)$ (или куда-нибудь ещё). В конце первого прохода получатся значения $|b_1 \dots b_n|_a$ для всех $a \in \Sigma$, имея которые, можно получить искомые суммы по всем $a < b$ для всех $b \in \Sigma$. На втором проходе эти суммы добавляются к текущим значениям $p(i)$, и получаются искомые значения перестановки. $\square$

Пример 4 (BWT^{-1}). Дана преобразованная строка $w' = bbbaaaaa$, а также номер строки 4. Сперва находится число вхождений каждого символа в каждый префикс, а на их основе — перестановка p .

	$b(1)$	$b(2)$	$b(3)$	$a(4)$	$a(5)$	$a(6)$	$a(7)$	$a(8)$
a	0	0	0	1	2	3	4	5
b	0	1	2	3	3	3	3	3
p	$1 + 0 + 5$	$1 + 1 + 5$	$1 + 2 + 5$	$1 + 0 + 0$	$1 + 1 + 0$	$1 + 2 + 0$	$1 + 3 + 0$	$1 + 4 + 0$

Отсюда $p(4) = 1$, $p(1) = 6$, $p(6) = 3$, $p(3) = 8$, $p(8) = 5$, $p(5) = 2$, $p(2) = 7$ (и далее $p(7) = 4$). Пользуясь этой перестановкой, можно восстановить исходную строку: $b_7 b_2 b_5 b_8 b_3 b_6 b_1 b_4 = abaababa$.

Применяя BWT для циклических сдвигов одной и той же строки порождает одну и ту же таблицу отсортированных циклических сдвигов.

Список литературы

- [1994] M. Burrows, D. J. Wheeler, “A block sorting lossless data compression algorithm”, TR 124, DEC Corporation, 1994.
- [1984] T. A. Welch, “A technique for high-performance data compression”, *IEEE Computer*, 17:6 (1984), 8–19.
- [1977] J. Ziv, A. Lempel, “A universal algorithm for sequential data compression”, *IEEE Transactions on Information Theory*, 23:3 (1977), 337–343.
- [1978] J. Ziv, A. Lempel, “Compression of individual sequences via variable-rate coding”, *IEEE Transactions on Information Theory*, 24:5 (1978), 530–536.